



PERTEMUAN II

Exploratory Data Analysis dan Data Preprocessing

2.1. TUJUAN PEMBELAJARAN

- A. Mahasiswa mampu memahami konsep dasar *Artificial Intelligence* (AI)
- B. Mahasiswa mengetahui dasar-dasar *Machine Learning* (ML)
- C. Mahasiswa dapat menerapkan langkah awal analisis data
- D. Mahasiswa mampu mengetahui dan menerapkan teknik-teknik dasar dalam *data preprocessing*

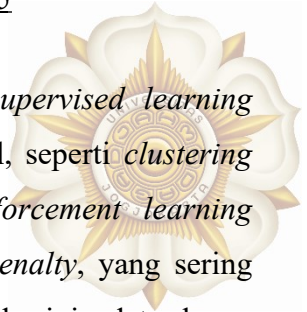
2.2. DASAR TEORI

- **Artificial Intelligence**

Artificial Intelligence (AI) merupakan bidang ilmu komputer yang berupaya membuat mesin mampu melakukan tugas-tugas yang biasanya membutuhkan kecerdasan manusia, seperti berpikir, belajar, mengenali pola, dan mengambil keputusan. AI tidak hanya terbatas pada sistem cerdas yang meniru manusia, tetapi juga mencakup algoritma dan pendekatan komputasional yang memungkinkan suatu sistem bekerja secara otonom dan adaptif. Perkembangan komputer modern, besarnya volume data, dan meningkatnya kemampuan komputasi membuat AI berkembang pesat di berbagai sektor seperti kesehatan, finansial, transportasi, hingga hiburan. Google AI Guides menjelaskan bahwa AI modern sangat bertumpu pada data, karena kemampuan kecerdasan sebuah sistem sangat bergantung pada kualitas data yang dipelajari dan pola-pola yang berhasil dipahami selama proses pemodelan.

- **Machine Learning**

Machine Learning adalah salah satu cabang AI yang menjadi fondasi bagi banyak inovasi teknologi saat ini. *Machine Learning* merupakan sebuah metode yang memungkinkan mesin meningkatkan performanya pada suatu tugas berdasarkan data. Inti dari ML adalah kemampuan sistem untuk mengenali pola dan membuat prediksi tanpa harus diprogram dengan aturan eksplisit. Secara umum, ML terbagi menjadi tiga, yaitu *supervised learning*, *unsupervised learning*, dan *reinforcement learning*. *Supervised learning* menggunakan data berlabel untuk melakukan prediksi atau klasifikasi, misalnya



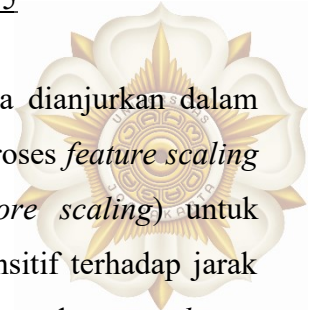
memprediksi harga rumah atau mendeteksi sentimen teks. *Unsupervised learning* berfungsi menemukan struktur tersembunyi dalam data tanpa label, seperti *clustering* pelanggan berdasarkan kebiasaan belanja. Sementara itu, *reinforcement learning* memungkinkan model belajar dari *feedback* berupa *reward* dan *penalty*, yang sering digunakan dalam robotika dan *game* AI. Dalam semua metode ini, data harus dipersiapkan dengan baik agar algoritma dapat belajar secara optimal. Kualitas model ML tidak hanya ditentukan oleh algoritma yang digunakan, tetapi juga oleh kualitas data dan *preprocessing* yang mendasarinya.

- **Exploratory Data Analysis**

Sebelum data dapat digunakan untuk membangun model ML, perlu dilakukan *Exploratory Data Analysis* (EDA). EDA merupakan proses penggalian informasi awal dari dataset untuk memahami karakteristik dan kondisi data secara menyeluruh. Gagasan EDA pertama kali diperkenalkan oleh John W. Tukey, yang menekankan pentingnya “mendengarkan apa yang data katakan” sebelum terburu-buru membuat asumsi atau membangun model. Dalam praktik modern, seperti yang dijelaskan dalam *Kaggle Learn* dan *IBM Data Analytics Guides*, EDA melibatkan berbagai aktivitas seperti melihat distribusi variabel melalui histogram atau *boxplot*, mencari korelasi antar fitur menggunakan *heatmap*, mendeteksi *outliers*, memeriksa tipe data, serta mengidentifikasi masalah seperti *missing values* atau duplikasi. Tahapan ini sangat penting karena memberikan gambaran awal tentang kualitas dataset, potensi masalah, dan tindakan apa saja yang perlu dilakukan selama *preprocessing*. Tanpa EDA, proses pemodelan bisa tidak akurat atau bias akibat ketidaktahuan terhadap struktur dan perilaku data.

- **Data Preprocessing**

Data preprocessing adalah proses sistematis untuk membersihkan, mengubah, dan menyiapkan data agar dapat digunakan secara optimal dalam algoritma *machine learning*. *Preprocessing* disebut sebagai tahapan kritis yang secara langsung memengaruhi performa dan generalisasi model. Tahapan pertama biasanya dimulai dari *data cleaning*, yaitu penanganan *missing values* menggunakan metode imputasi seperti *mean*, median, atau mode, serta menghapus atau memperbaiki data duplikat yang dapat mengganggu konsistensi analisis. Tahap berikutnya adalah data *transformation*, yang bertujuan mengubah data ke bentuk yang lebih representatif. Fitur kategorikal perlu di-*encode*



menggunakan *label encoding* atau *one-hot encoding*, sebagaimana dianjurkan dalam *Google Machine Learning Crash Course*. Selanjutnya, dilakukan proses *feature scaling* seperti normalisasi (*min-max scaling*) atau standardisasi (*z-score scaling*) untuk menyamakan rentang nilai antar fitur, terutama pada algoritma sensitif terhadap jarak seperti KNN dan SVM. Langkah penting lainnya adalah melakukan dataset *splitting* seperti *train-test split* (misalnya 80:20) untuk memastikan model dapat diuji secara objektif dan tidak *overfitting*. Seluruh proses ini umumnya dilakukan menggunakan pustaka numerik dan analitik seperti NumPy dan Pandas.

2.3. ALAT DAN BAHAN

- **Perangkat Keras**
 - Komputer/Laptop
- **Perangkat Lunak**
 - Python3
 - Google Colaboratory

2.4. LANGKAH PERCOBAAN

- **Exploratory Data Analysis (EDA)**

1. Impor Pustaka

```
import pandas as pd
import matplotlib.pyplot as plt
import numpy as np
```

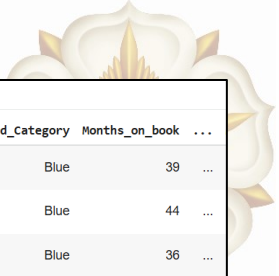
Gambar 2.4.1 Import Library

2. Load dataset

Link dataset: <https://www.kaggle.com/datasets/sakshigoyal7/credit-card-customers>

```
df_churning = pd.read_csv('https://raw.githubusercontent.com/ganjar87/
data_science_practice/main/BankChurners.csv', delimiter=',')
```

3. Melihat lima baris pertama dengan df.head()



```
df_churning.head()
```

	CLIENTNUM	Attrition_Flag	Customer_Age	Gender	Dependent_count	Education_Level	Marital_Status	Income_Category	Card_Category	Months_on_book	...
0	768805383	Existing Customer	45	M	3	High School	Married	\$60K - \$80K	Blue	39	...
1	818770008	Existing Customer	49	F	5	Graduate	Single	Less than \$40K	Blue	44	...
2	713982108	Existing Customer	51	M	3	Graduate	Married	\$80K - \$120K	Blue	36	...
3	769911858	Existing Customer	40	F	4	High School	Unknown	Less than \$40K	Blue	34	...
4	709106358	Existing Customer	40	M	3	Uneducated	Married	\$60K - \$80K	Blue	21	...

5 rows x 21 columns

Gambar 2.4.2 Lima Baris Pertama Dataset

4. Melihat lima baris terakhir dengan df.tail()

```
df_churning.tail()
```

	CLIENTNUM	Attrition_Flag	Customer_Age	Gender	Dependent_count	Education_Level	Marital_Status	Income_Category	Card_Category	Months_on_book	...
10122	772366833	Existing Customer	50	M	2	Graduate	Single	\$40K - \$60K	Blue	40	...
10123	710638233	Attrited Customer	41	M	2	Unknown	Divorced	\$40K - \$60K	Blue	25	...
10124	716506083	Attrited Customer	44	F	1	High School	Married	Less than \$40K	Blue	36	...
10125	717406983	Attrited Customer	30	M	2	Graduate	Unknown	\$40K - \$60K	Blue	36	...
10126	714337233	Attrited Customer	43	F	2	Graduate	Married	Less than \$40K	Silver	25	...

5 rows x 21 columns

Gambar 2.4.3 Lima Baris Terakhir Dataset

5. Menyimpan nilai dari class menjadi 2 dataframe yang berbeda



```
existing_data = df_churning[(df_churning.Attrition_Flag == 'Existing Customer')]
attrited_data = df_churning[(df_churning.Attrition_Flag == 'Attrited Customer')]
```

Gambar 2.4.4 Pemisahan Dataset Bank Churner

Kode tersebut merupakan kode untuk memisahkan dataset berdasarkan nilai kolom `Attrition_Flag`, sehingga terbentuk dua subset yaitu *existing_data* yang berisi pelanggan yang masih aktif (*Existing Customer*) dan *attrited_data* yang berisi pelanggan yang berhenti atau *churn* (*Attrited Customer*).

6. Line chart



```
#sumbu x
x = np.array([2, 4, 6, 8])

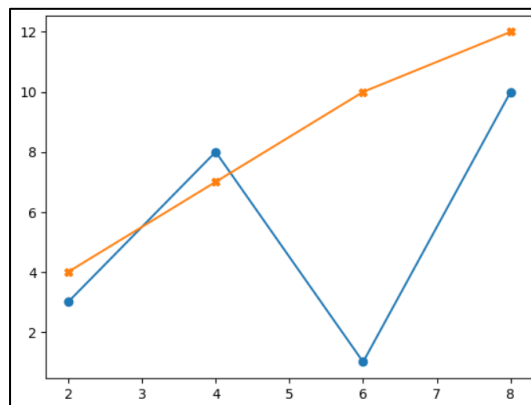
#sumbu y
y1 = np.array([3, 8, 1, 10])
y2 = np.array([4, 7, 10, 12])

plt.plot(x, y1, marker = 'o')
plt.plot(x, y2, marker = 'x')

plt.show()
```

Gambar 2.4.5 Pembuatan Line Chart dengan Data Dummy

Pada kode di atas, x dipakai sebagai sumbu horizontal, y1 dan y2 sebagai dua garis berbeda di sumbu vertikal. Parameter **marker='o'** dan **'x'** digunakan untuk memberi tanda di setiap titik data. **plt.show()** digunakan untuk menampilkan plot ke layar. Tujuan pemanggilan fungsi ini adalah menunjukkan cara membuat *line chart* untuk melihat tren atau perubahan nilai dari dua seri data sekaligus. *Line chart* yang ditampilkan dapat dilihat pada Gambar 2.4.6.



Gambar 2.4.6 Line Chart dari Data Dummy

Setelah memahami konsep *line chart* menggunakan data *dummy*, langkah berikutnya adalah menerapkannya pada dataset IoT untuk menampilkan perubahan nilai sensor terhadap waktu.

```
df_iot = pd.read_csv('https://raw.githubusercontent.com/ganjar87/data_science_practice/main/datatraining.txt')
df_iot['date'] = pd.to_datetime(df_iot['date'])
df_iot.head()
```

Gambar 2.4.7 Load Dataset IoT

Setelah dataset siap, `df_iot.head()` dipanggil untuk menampilkan beberapa baris awal sebagai verifikasi bahwa data telah terbaca dengan benar.



	date	Temperature	Humidity	Light	CO2	HumidityRatio	Occupancy
1	2015-02-04 17:51:00	23.18	27.2720	426.0	721.25	0.004793	1
2	2015-02-04 17:51:59	23.15	27.2675	429.5	714.00	0.004783	1
3	2015-02-04 17:53:00	23.15	27.2450	426.0	713.50	0.004779	1
4	2015-02-04 17:54:00	23.15	27.2000	426.0	708.25	0.004772	1
5	2015-02-04 17:55:00	23.10	27.2000	426.0	704.50	0.004757	1

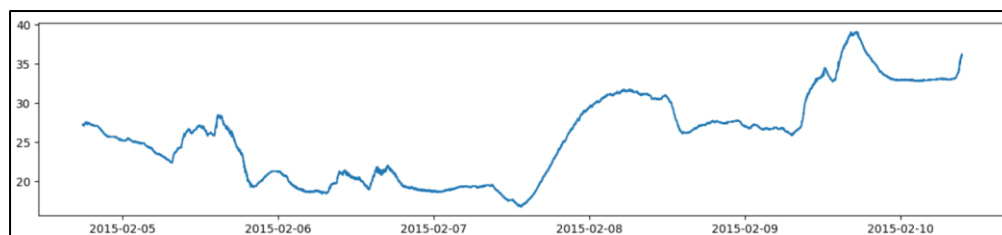
Gambar 2.4.8 Sampel Data IoT

Langkah berikutnya adalah menampilkan *line chart* untuk memvisualisasikan perubahan nilai sensor terhadap waktu.

```
plt.figure(figsize=(15, 3))
plt.plot(df_iot['date'], df_iot['Humidity'])
plt.show()
```

Gambar 2.4.9 Pembuatan Line Chart dengan Data IoT

Kode `plt.figure(figsize=(15, 3))` berfungsi untuk mengatur ukuran grafik agar panjang secara horizontal. Selanjutnya digunakan `plt.plot(df_iot['date'], df_iot['Humidity'])` untuk memetakan kolom *date* sebagai sumbu-x dan nilai *Humidity* sebagai sumbu-y. Grafik kemudian ditampilkan dengan `plt.show()`.



Gambar 2.4.10 Line Chart dengan Data IoT

7. Pie chart

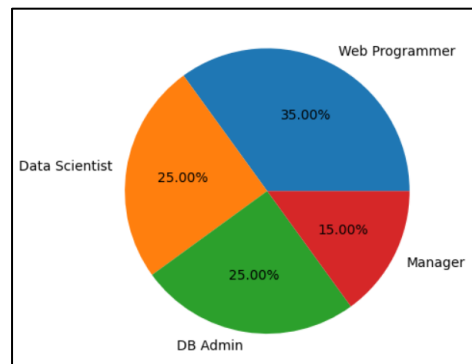


```
y = np.array([35, 25, 25, 15])
mylabels = ["Web Programmer", "Data Scientist", "DB Admin", "Manager"]

plt.pie(y, labels = mylabels, autopct='% .2f%%')
plt.show()
```

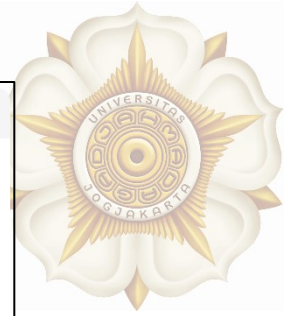
Gambar 2.4.11 Pembuatan Pie Chart dengan Data Dummy

Pembuatan *pie chart* dimulai dengan mendefinisikan *array* *y* yang berisi nilai-nilai numerik mewakili frekuensi tiap kategori, yaitu [35, 25, 25, 15]. Kemudian dibuat daftar **mylabels** yang merupakan nama kategori: “Web Programmer”, “Data Scientist”, “DB Admin”, dan “Manager”. Fungsi **plt.pie()** digunakan untuk menghasilkan *pie chart*, dengan parameter **labels=mylabels** untuk memberi nama pada setiap irisan grafik. Sedangkan **autopct='% .2f%%'** digunakan untuk menampilkan persentase masing-masing irisan dengan format dua angka di belakang koma. Terakhir, **plt.show()** memunculkan visualisasi ke layar.



Gambar 2.4.12 Pie Chart dengan Data Dummy

Selanjutnya, pembuatan *pie chart* dilakukan menggunakan dataset sebelumnya yaitu *Bank Churner*. Visualisasi ini bertujuan untuk menampilkan proporsi kategori pada variabel **Marital_Status**.



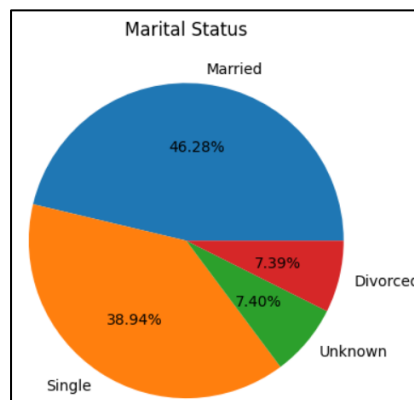
```
data = df_churning['Marital_Status'].value_counts()

# fungsi index untuk melihat label dari series
label = data.index

plt.pie(data, labels=label, autopct='%.2f%%')
plt.title('Marital Status')
plt.show()
```

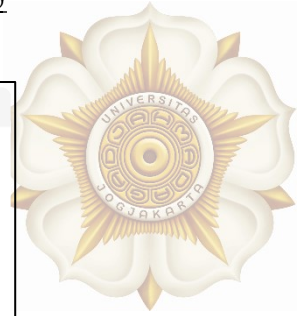
Gambar 2.4.13 Pembuatan Pie Chart dengan Data Bank Churner

Kode dimulai dengan memanggil `df_churning['Marital_Status'].value_counts()` untuk menghitung jumlah kemunculan setiap kategori dalam kolom tersebut. Nilai frekuensi yang dihasilkan kemudian disimpan dalam variabel `data`, sementara `data.index` digunakan untuk mengambil nama kategorinya dan disimpan ke dalam variabel `label`. Fungsi `plt.pie(data, labels=label, autopct='%.2f%%')` digunakan untuk menggambar *pie chart* dengan persentase setiap kategori ditampilkan secara otomatis. Selanjutnya, `plt.title('Marital Status')` digunakan untuk menambahkan judul pada grafik.



Gambar 2.4.14 Pie Chart Marital Status

8. Bar plot



```

y = np.array([35, 25, 25, 15])
x = ["Web Programmer", "Data Scientist", "DB Admin", "Manager"]

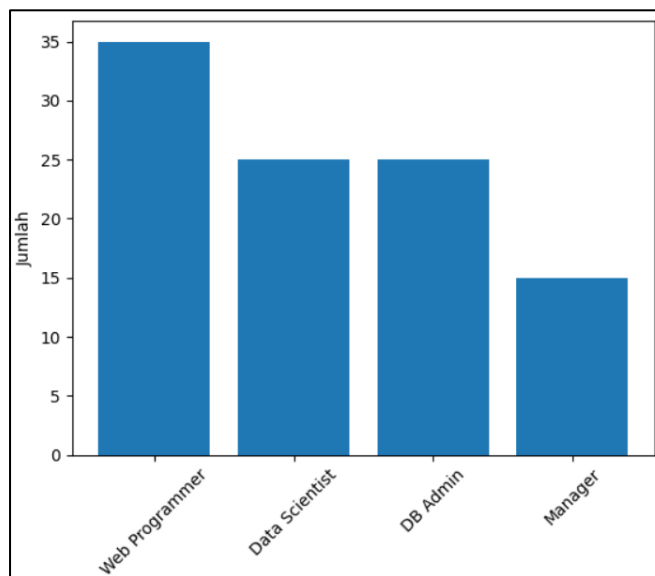
plt.bar(x,y)

plt.xticks(rotation=45)
plt.ylabel('Jumlah')
plt.show()

```

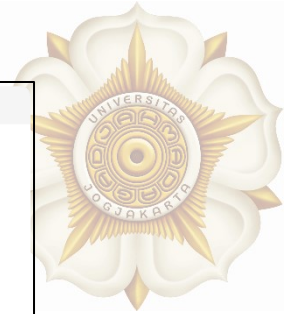
Gambar 2.4.15 Pembuatan Bar Plot dengan Data Dummy

Pembuatan bar plot dimulai dengan mendefinisikan *array* *y* yang berisi nilai jumlah pekerja pada beberapa jenis profesi, dan daftar *x* yang berisi nama kategori profesi seperti “Web Programmer”, “Data Scientist”, “DB Admin”, dan “Manager”. Selanjutnya, fungsi `plt.bar(x, y)` digunakan untuk menggambar batang pada sumbu *x* berdasarkan kategori, dengan tinggi batang mengikuti nilai yang diberikan pada variabel *y*.



Gambar 2.4.16 Bar Plot dengan Data Dummy

Langkah selanjutnya adalah membuat bar plot dengan dataset *Bank Churner* untuk membandingkan distribusi kategori **Marital_Status** antara dua kelompok nasabah, yaitu *existing customers* dan *attrited customers*.



```

y1 = existing_data['Marital_Status'].value_counts()
y2 = attrited_data['Marital_Status'].value_counts()

x1 = np.arange(len(y1.index))
x2 = np.arange(len(y2.index))
bar_width = 0.4

plt.bar(x1, y1, width=bar_width)
plt.bar(x2+bar_width, y2, width=bar_width)

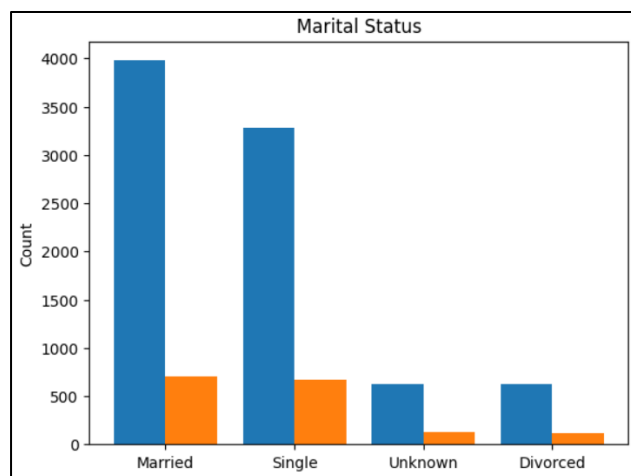
plt.title('Marital Status')
plt.ylabel('Count')
plt.xticks(x2 + bar_width / 2, labels=df_churning.Marital_Status.unique())

plt.show()

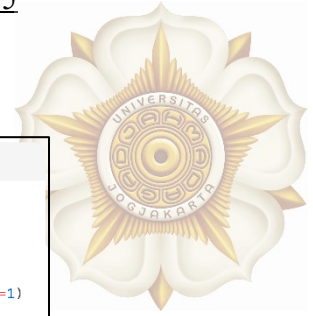
```

Gambar 2.4.17 Pembuatan Bar Plot Marital Status

Langkah pertama adalah menghitung jumlah masing-masing kategori marital status pada kedua subset menggunakan `value_counts()`, yang disimpan dalam variabel `y1` dan `y2`. Lalu membuat indeks posisi awal masing-masing kategori dengan `np.arange(len(y1.index))` dan `np.arange(len(y2.index))`. Variabel `bar_width = 0.4` digunakan untuk menentukan lebar batang. Fungsi `plt.bar(x1, y1, width=bar_width)` menggambar batang pertama untuk *existing customers*, sementara `plt.bar(x2 + bar_width, y2, width=bar_width)` menggambar batang kedua untuk *attrited customers*. Selanjutnya, judul grafik ditetapkan dengan `plt.title("Marital Status")`, dan sumbu-y diberi label "Count". `plt.xticks(x2 + bar_width / 2, labels = df_churning.Marital_Status.unique())`, digunakan untuk memastikan label kategori muncul tepat di tengah pasangan batang untuk setiap kelompok marital status.



Gambar 2.4.18 Bar Plot Marital Status



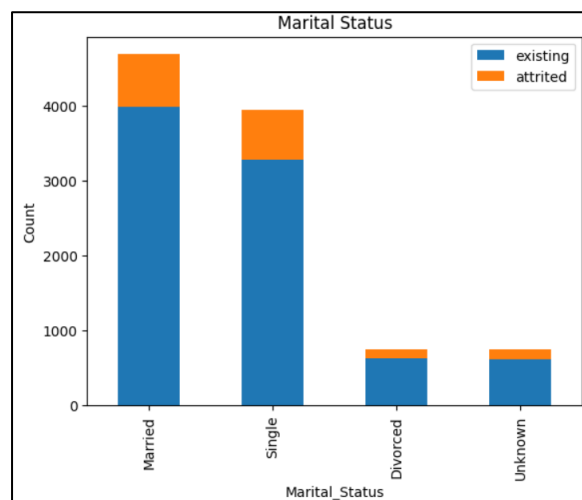
9. *Stacked* bar plot

```
# menggunakan dataset diatas
# contoh menggunakan stacked bar plot
data1 = existing_data['Marital_Status'].value_counts()
data2 = attrited_data['Marital_Status'].value_counts()
df_new = pd.concat([data1, data2], keys=['existing', 'attrited'], axis=1)

#df_new.plot.bar() #bisa pakai cara 1
df_new.plot(kind='bar', stacked=True) # bisa pakai cara 2
plt.title('Marital Status')
plt.ylabel('Count')
plt.show()
```

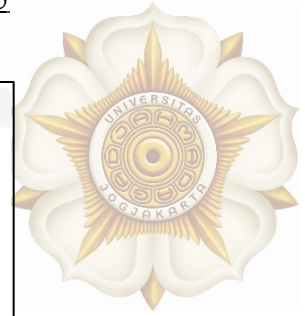
Gambar 2.4.19 Pembuatan *Stacked Bar Plot* Marital Status

Langkah selanjutnya yaitu membuat *stacked* bar plot untuk menganalisis komposisi kategori **Marital_Status** pada dua kelompok nasabah dalam dataset *Bank Churner*. Jumlah masing-masing kategori marital status dihitung menggunakan `value_counts()`, yang disimpan dalam variabel `data1` untuk *existing customers* dan `data2` untuk *attrited customers*, kemudian digabungkan menjadi satu *DataFrame* menggunakan `pd.concat()` dengan parameter `keys=['existing', 'attrited']` dan `axis=1`, sehingga muncul dua kolom baru yang merepresentasikan kedua kelompok tersebut. Visualisasi dibuat menggunakan `df_new.plot(kind='bar', stacked=True)`, yang menghasilkan diagram batang bertumpuk



Gambar 2.4.20 *Stacked Bar Plot* Marital Status

10. Histogram

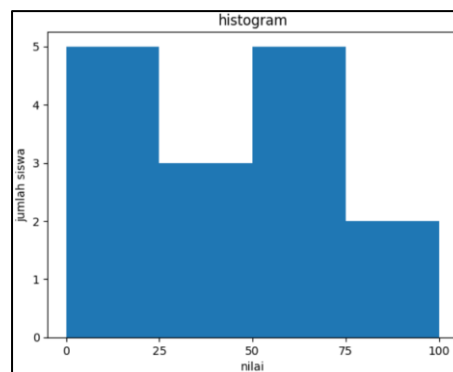


```
data = np.array([22,87,5,43,56,73,55,54,11,20,51,5,79,31,27])
bins = [0,25,50,75,100]

plt.hist(data, bins = bins)
plt.title("histogram")
plt.xticks([0,25,50,75,100])
plt.xlabel('nilai')
plt.ylabel('jumlah siswa')
plt.show()
```

Gambar 2.4.21 Pembuatan Histogram dengan Data Dummy

Pembuatan histogram dimulai dengan pembuatan data *dummy* yang disimpan dalam *array*. Variabel *bins* didefinisikan digunakan untuk membagi data ke dalam empat interval atau kelas. Fungsi `plt.hist(data, bins=bins)` digunakan untuk menggambar histogram, di mana setiap batang menunjukkan jumlah data yang jatuh ke dalam masing-masing interval.



Gambar 2.4.22 Histogram dengan Data Dummy

Langkah selanjutnya adalah membuat histogram dengan dataset *Bank Churner*, khususnya pada variabel **Customer_Age**.

```
data1 = existing_data['Customer_Age']
data2 = attrited_data['Customer_Age']

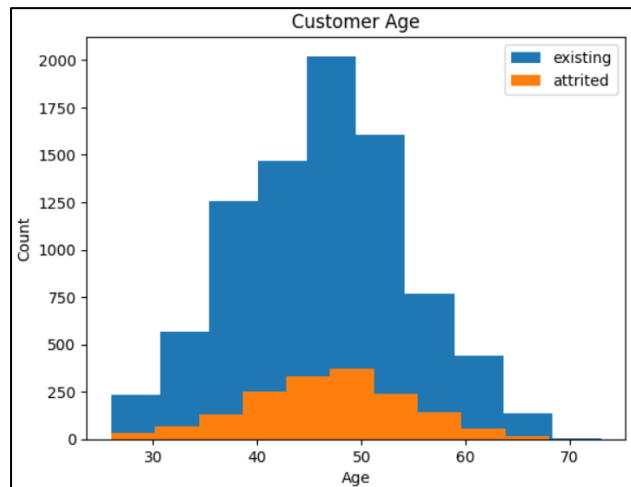
plt.hist(data1, label='existing')
plt.hist(data2, label='attrited')

plt.title('Customer Age')
plt.ylabel('Count')
plt.xlabel('Age')

plt.legend()
plt.show()
```

Gambar 2.4.23 Pembuatan Histogram Customer Age

Dua subset data *existing_data* dan *attrited_data* digunakan untuk memisahkan nasabah yang masih aktif dan nasabah yang sudah berhenti. Nilai usia dari masing-masing kelompok diambil menggunakan `existing_data['Customer_Age']` dan `attrited_data['Customer_Age']`, lalu disimpan dalam `data1` dan `data2`. Visualisasi dilakukan dengan memanggil `plt.hist(data1, label='existing')` dan `plt.hist(data2, label='attrited')`.



Gambar 2.4.24 Histogram Customer Age

X

11. Scatter plot

```

y1 = attrited_data['Months_on_book']
x1 = attrited_data['Customer_Age']

y2 = existing_data['Months_on_book']
x2 = existing_data['Customer_Age']

plt.scatter(x2, y2)
plt.scatter(x1, y1)

plt.title('Customer age vs months on book')
plt.ylabel('Months on book')
plt.xlabel('Customer age')

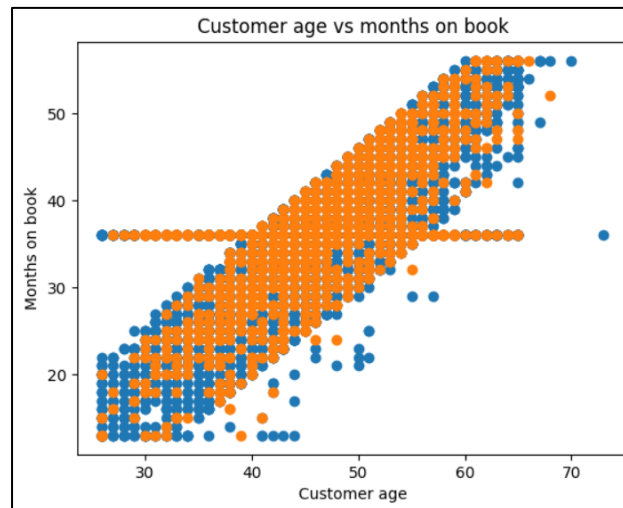
plt.show()

```

Gambar 2.4.25 Pembuatan Scatter Plot dengan `plt.scatter`

Pembuatan *scatter* plot dimulai dengan memisahkan kedua variabel untuk dua kelompok yaitu *attrited customers* (`x1` untuk usia dan `y1` untuk lama berlangganan) serta *existing customers* (`x2` dan `y2`). Dengan menggunakan `plt.scatter(x2, y2)`

dan `plt.scatter(x1, y1)`, kedua kelompok digambarkan dalam satu grafik *scatter*.



Gambar 2.4.26 Scatter Plot Customer Age vs Months on Book

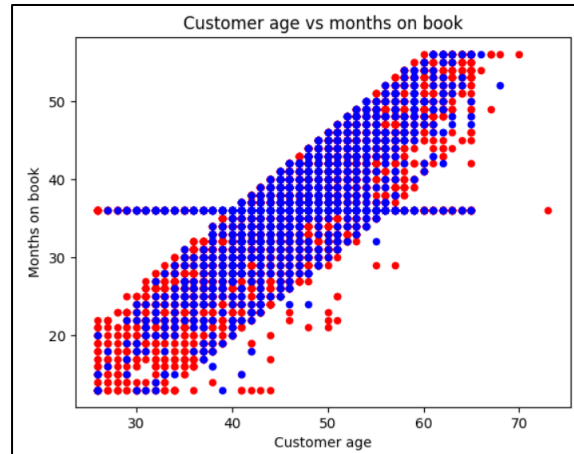
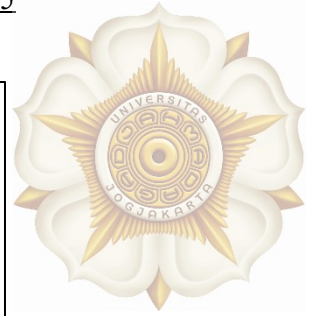
Versi lain pembuatan *scatter* plot menggunakan fungsi `.plot.scatter()`. Cara ini lebih ringkas dan terstruktur dibandingkan pemanggilan `plt.scatter()` secara manual.

```
# menggunakan dataset diatas
# versi 2. contoh menggunakan fungsi plot.scatter dari object dataframe
ax=existing_data.plot.scatter(x='Customer_Age', y='Months_on_book', c='red')
attrited_data.plot.scatter(x='Customer_Age', y='Months_on_book', c='blue', ax=ax)

plt.title('Customer age vs months on book')
plt.ylabel('Months on book')
plt.xlabel('Customer age')
plt.show()
```

Gambar 2.4.27 Pembuatan Scatter Plot dengan fungsi `plot.scatter`

Visualisasi dimulai dengan memanggil `existing_data.plot.scatter(x='Customer_Age', y='Months_on_book', c='red')`, yang menghasilkan scatter plot berwarna merah untuk nasabah yang masih aktif. Hasil plot ini disimpan dalam variabel `ax`, yang berfungsi sebagai *axes reference* untuk menumpuk plot berikutnya pada grafik yang sama. Baris selanjutnya memanggil `attrited_data.plot.scatter(x='Customer_Age', y='Months_on_book', c='blue', ax=ax)`, yang menggambar titik-titik untuk nasabah churn dengan warna biru pada axis yang sama.



Gambar 2.4.28 Scatter Plot Customer Age vs Months on Book

12. Box plot

```
# menggunakan dataset diatas
# versi 1. contoh menggunakan fungsi boxplot dari plt. single group.
plt.boxplot(df_churning['Customer_Age'])

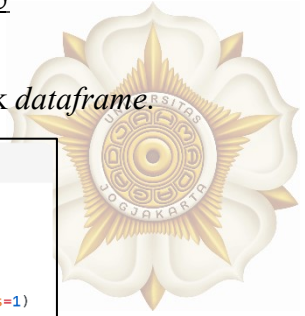
plt.title('Customer age')
plt.ylabel('age')
plt.xticks([1], labels=['all customer'])
plt.show()
```

Gambar 2.4.29 Pembuatan Box Plot dengan plt.boxplot

Pembuatan *boxplot* dilakukan dengan memanggil `plt.boxplot(df_churning['Customer_Age'])`. Visualisasi yang dihasilkan memperlihatkan nilai median, sebaran kuartil, serta potensi *outlier* dalam satu kelompok data.



Gambar 2.4.30 Box Plot Customer Age



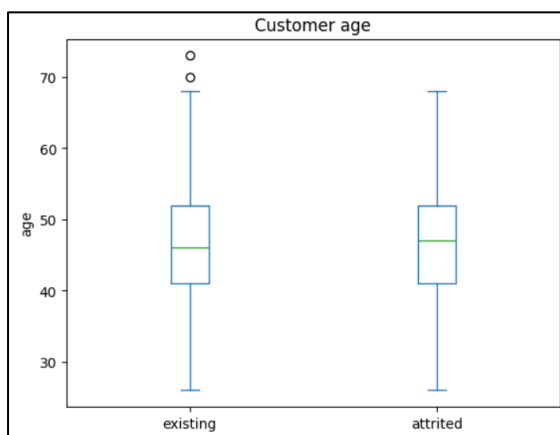
Pembuatan *boxplot* menggunakan fungsi `plot.box()` dari objek *dataframe*.

```
# menggunakan dataset diatas
# versi 2. contoh menggunakan fungsi plot.box dari object dataframe
data1 = existing_data['Customer_Age']
data2 = attrited_data['Customer_Age']
df_new = pd.concat([data1, data2], keys=['existing', 'attrited'], axis=1)

df_new.plot.box() #bisa pakai cara ini
#df_new.plot(kind='box') #bisa menggunakan cara ini juga

plt.title('Customer age')
plt.ylabel('age')
plt.show()
```

Gambar 2.4.31 Pembuatan Box Plot dengan fungsi `plot.box`



Gambar 2.4.32 Box Plot Customer Age

13. Violin plot

```
# menggunakan dataset diatas
# versi 1. contoh menggunakan fungsi violinplot dari plt. multiple group.

data1 = existing_data['Customer_Age'].values
data2 = attrited_data['Customer_Age'].values
#buat object dictionary dahulu
my_dictionary = {'existing': data1, 'attrited': data2}
#ambil value nya
plt.violinplot(list(my_dictionary.values()))

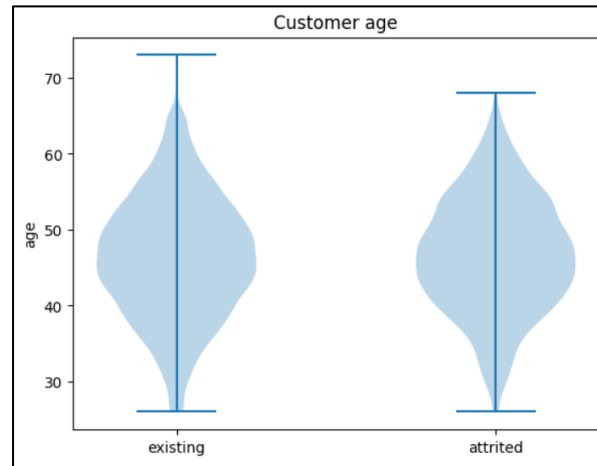
plt.title('Customer age')
plt.ylabel('age')
plt.xticks([1,2], labels=list(my_dictionary.keys()))
plt.show()
```

Gambar 2.4.33 Pembuatan Violin Plot

Pembuatan violin plot dimulai dengan mengambil nilai `Customer_Age` dari kedua kelompok pelanggan, lalu menyusunnya dalam *dictionary*. Data tersebut kemudian divisualisasikan menggunakan fungsi utama `plt.violinplot()`, yang



menampilkan distribusi dan kepadatan usia untuk *existing* dan *attrited customers* dalam satu grafik.



Gambar 2.4.34 Violin Plot Customer Age

14. Sub plot

```
#versi 1
plt.figure(figsize=(10, 10))
plt.subplot(2, 2, 1) #2 baris, 2 kolom, plot 1
plt.boxplot(existing_data['Customer_Age'])
plt.ylabel('age')
plt.xticks([1], labels=['existing customer'])

plt.subplot(2, 2, 2)
plt.boxplot(attrited_data['Customer_Age'])
plt.ylabel('age')
plt.xticks([1], labels=['attrited customer'])

plt.subplot(2, 2, 3)
existing_data['Customer_Age'].plot.hist()
plt.ylabel('Count')
plt.xlabel('Age of existing customer')
plt.ylim([0, 2100])

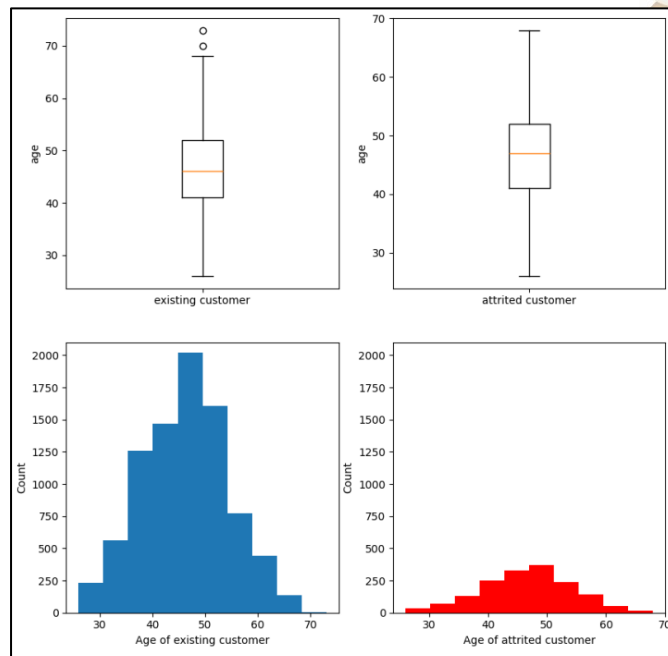
plt.subplot(2, 2, 4)
attrited_data['Customer_Age'].plot.hist(color='r')
plt.ylabel('Count')
plt.xlabel('Age of attrited customer')
plt.ylim([0, 2100])

plt.show()
```

Gambar 2.4.35 Pembuatan Sub Plot

Kode di atas membuat empat visualisasi sekaligus dalam satu *figure* menggunakan `plt.subplot()`, yaitu dua *boxplot* (untuk *existing* dan *attrited customers*) serta dua histogram usia untuk kedua kelompok tersebut. Tujuan penggunaan *subplot* adalah

menampilkan perbandingan distribusi usia antar kelompok dalam satu tampilan sehingga memudahkan analisis visual secara menyeluruh.



Gambar 2.4.36 Sub Plot

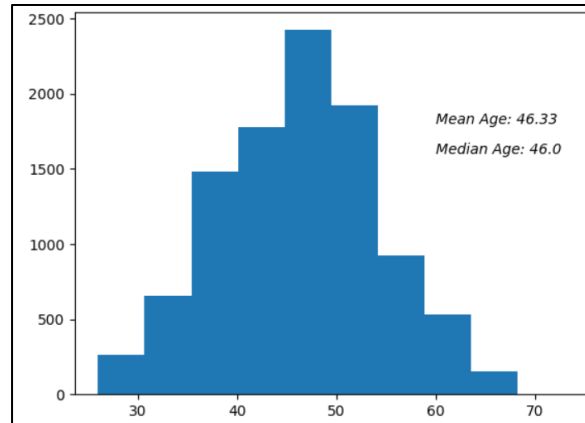
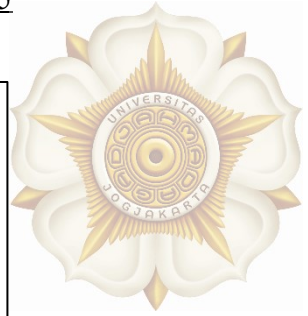
15. Annotation

```
# Show the distribution of a data and its measure of central tendency
data = df_churning['Customer_Age']
plt.hist(data)

plt.text(60, 1600, "Median Age: " + str(round(data.median(),2)),
        style = 'italic', fontsize=10)
plt.text(60, 1800, "Mean Age: " + str(round(data.mean(),2)),
        style = 'italic', fontsize=10)
plt.show()
```

Gambar 2.4.37 Pembuatan Annotation

Kode di atas menampilkan histogram usia pelanggan (Customer_Age) dan menambahkan informasi statistik langsung pada grafik menggunakan `plt.text()`. Dua anotasi yang ditampilkan yaitu *Median Age* dan *Mean Age*.



Gambar 2.4.38 Annotation pada Histogram

16. Axis

```
#x = np.array([1, 2, 3, 4])
x = np.array(["01/02/2020", "01/03/2020", "01/04/2020", "01/05/2020"])
y1 = np.array([3, 8, 1, 10])
y2 = np.array([13, 4, 10, 12])

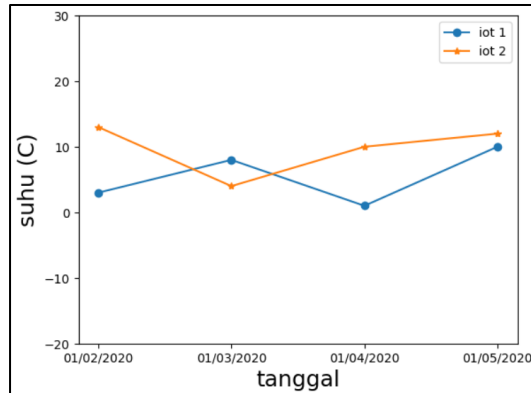
plt.plot(x,y1, marker='o', label='iot 1')
plt.plot(x,y2, marker='*', label='iot 2')
plt.legend()

# set label
plt.ylabel('suhu (c)', fontsize=18) # buat label untuk sumbu y
plt.xlabel('tanggal', fontsize=18) # buat label untuk sumbu x

#set axis range
plt.ylim([-20,30])
plt.show()
```

Gambar 2.4.39 Pengaturan Axis

Kode di atas menampilkan dua *line chart* yang menunjukkan data suhu pada beberapa hari. Fungsi `plt.plot()` digunakan untuk menggambar dua garis berbeda dengan *marker* serta label yang dibedakan. Kemudian `plt.legend()` ditambahkan untuk menampilkan keterangan setiap garis. Setelah itu, `plt.ylabel()` dan `plt.xlabel()` digunakan untuk memberikan label pada sumbu-y dan sumbu-x agar grafik lebih informatif. `plt.ylim([-20, 30])` digunakan untuk mengatur rentang nilai pada sumbu-y, agar tampilan grafik menjadi lebih mudah dibaca.



Gambar 2.4.40 Line Chart

17. Legend

```
data1 = existing_data['Customer_Age']
data2 = attrited_data['Customer_Age']

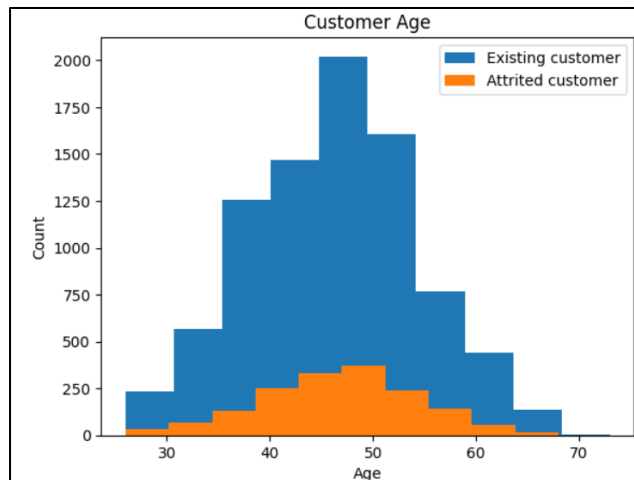
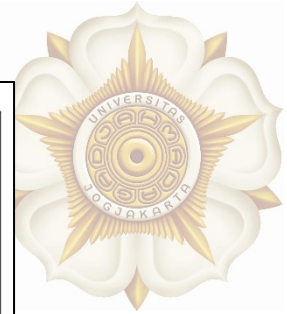
plt.hist(data1, label='Existing customer') # pertama, set label dulu
plt.hist(data2, label='Attrited customer')

plt.title('Customer Age')
plt.ylabel('Count')
plt.xlabel('Age')

plt.legend() # kedua, panggil fungsi legend
plt.show()
```

Gambar 2.4.41 Pembuatan Legend

Kode di atas menampilkan dua histogram usia (Customer_Age) untuk *existing* dan *attrited customers* dalam satu grafik. Label untuk masing-masing kelompok ditentukan terlebih dahulu melalui parameter label pada fungsi `plt.hist()`. Setelah kedua histogram didefinisikan, `plt.legend()` dipanggil untuk menampilkan kotak *legend* yang menunjukkan warna dari masing-masing kelompok.



Gambar 2.4.42 Histogram dengan Legend

• Data Preprocessing

1. Impor pustaka

```
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.preprocessing import OneHotEncoder
from sklearn.preprocessing import LabelEncoder
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.preprocessing import OneHotEncoder
from sklearn.preprocessing import OrdinalEncoder
from sklearn.preprocessing import MinMaxScaler
from sklearn.preprocessing import StandardScaler
```

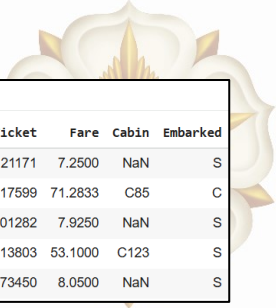
Gambar 2.4.43 Import Library

2. Load dataset

Link: <https://www.kaggle.com/datasets/yasserh/titanic-dataset>

```
df = pd.read_csv('https://raw.githubusercontent.com/ganjar87/
data_science_practice/main/train.csv')
```

3. Melihat lima baris data pertama dengan df.head()



```
df.head()
```

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500	NaN	S
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.2833	C85	C
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9250	NaN	S
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.1000	C123	S
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8.0500	NaN	S

Gambar 2.4.44 Lima Baris Pertama Dataset

4. Melihat lima baris data terakhir dengan df.tail()

```
df.tail()
```

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
886	887	0	2	Montvila, Rev. Juozas	male	27.0	0	0	211536	13.00	NaN	S
887	888	1	1	Graham, Miss. Margaret Edith	female	19.0	0	0	112053	30.00	B42	S
888	889	0	3	Johnston, Miss. Catherine Helen "Carrie"	female	NaN	1	2	W./C. 6607	23.45	NaN	S
889	890	1	1	Behr, Mr. Karl Howell	male	26.0	0	0	111369	30.00	C148	C
890	891	0	3	Dooley, Mr. Patrick	male	32.0	0	0	370376	7.75	NaN	Q

Gambar 2.4.45 Lima Baris Terakhir Dataset

5. Melihat bentuk dataset dengan df.shape

```
# Melihat bentuk dataset. Baris, kolom
df.shape

(891, 12)
```

Gambar 2.4.46 Melihat Bentuk Dataset dengan df.shape

Output (891, 12) menunjukkan bahwa dataset memiliki 891 baris data dan 12 kolom fitur.

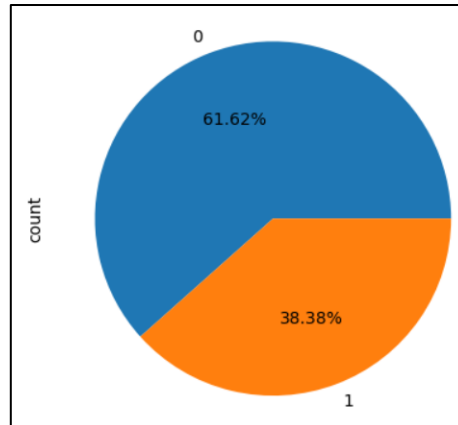
6. Visualisasi distribusi data



```
data = df['Survived'].value_counts()
data.plot(kind='pie', autopct='%0.2f%%')
plt.show()
```

Gambar 2.4.47 Membuat Pie Chart

Kode di atas membuat *pie chart* untuk menampilkan proporsi penumpang yang selamat dan tidak selamat berdasarkan nilai *Survived* dalam dataset. Hasilnya dapat dilihat pada Gambar 2.4.48

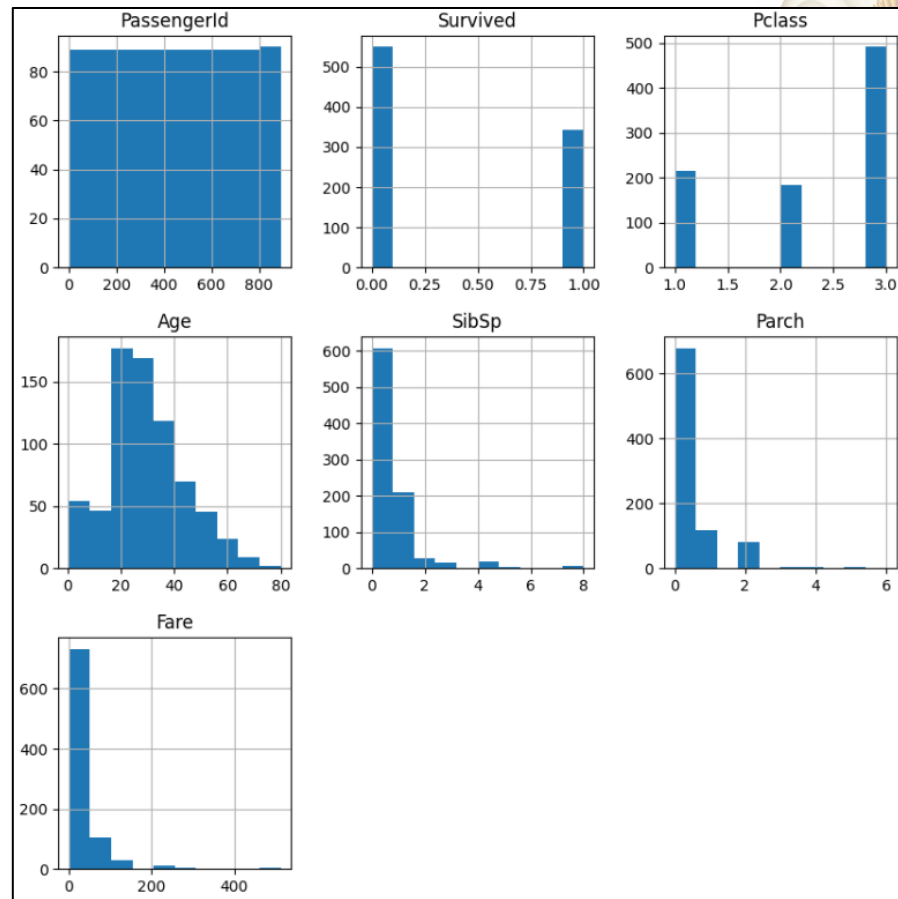


Gambar 2.4.48 Pie Chart Distribusi Data

```
df.hist(figsize=(10,10))
plt.show()
# menggunakan histogram, untuk melihat distribusi nya
```

Gambar 2.4.49 Membuat Histogram

Kode di atas menampilkan histogram untuk seluruh kolom numerik dalam dataset guna melihat distribusi datanya. Hasilnya dapat dilihat pada Gambar 2.4.50.



Gambar 2.4.50 Histogram Kolom Numerik

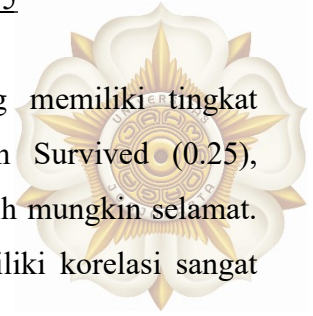
7. Analisis korelasi antar variabel

```
df.corr(numeric_only = True)
```

	PassengerId	Survived	Pclass	Age	SibSp	Parch	Fare
PassengerId	1.000000	-0.005007	-0.035144	0.036847	-0.057527	-0.001652	0.012658
Survived	-0.005007	1.000000	-0.338481	-0.077221	-0.035322	0.081629	0.257307
Pclass	-0.035144	-0.338481	1.000000	-0.369226	0.083081	0.018443	-0.549500
Age	0.036847	-0.077221	-0.369226	1.000000	-0.308247	-0.189119	0.096067
SibSp	-0.057527	-0.035322	0.083081	-0.308247	1.000000	0.414838	0.159651
Parch	-0.001652	0.081629	0.018443	-0.189119	0.414838	1.000000	0.216225
Fare	0.012658	0.257307	-0.549500	0.096067	0.159651	0.216225	1.000000

Gambar 2.4.51 Korelasi Antar Variabel

Kode di atas menghitung korelasi antar variabel numerik dalam dataset, sehingga kita bisa melihat hubungan mana yang paling kuat atau lemah antar fitur. Hasilnya, menunjukkan bahwa Pclass memiliki korelasi negatif cukup kuat dengan Survived



(-0.33), artinya penumpang kelas lebih rendah cenderung memiliki tingkat keselamatan lebih rendah. Fare berkorelasi positif dengan Survived (0.25), menunjukkan bahwa penumpang dengan tiket lebih mahal lebih mungkin selamat. Sementara variabel lain seperti Age, SibSp, dan Parch memiliki korelasi sangat lemah terhadap Survived.

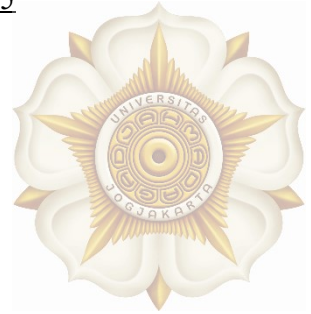
8. Descriptive statistics

df.describe()							
	PassengerId	Survived	Pclass	Age	SibSp	Parch	Fare
count	891.000000	891.000000	891.000000	714.000000	891.000000	891.000000	891.000000
mean	446.000000	0.383838	2.308642	29.699118	0.523008	0.381594	32.204208
std	257.353842	0.486592	0.836071	14.526497	1.102743	0.806057	49.693429
min	1.000000	0.000000	1.000000	0.420000	0.000000	0.000000	0.000000
25%	223.500000	0.000000	2.000000	20.125000	0.000000	0.000000	7.910400
50%	446.000000	0.000000	3.000000	28.000000	0.000000	0.000000	14.454200
75%	668.500000	1.000000	3.000000	38.000000	1.000000	0.000000	31.000000
max	891.000000	1.000000	3.000000	80.000000	8.000000	6.000000	512.329200

Gambar 2.4.52 Deskripsi Statistik

Hasil `df.describe()` menampilkan ringkasan statistik untuk fitur numerik. Dari output di atas, terlihat bahwa rata-rata usia penumpang adalah 29.7 tahun, dengan rentang dari 0.42 hingga 80 tahun. Harga tiket (Fare) memiliki variasi besar, mulai dari 0 hingga 512, menunjukkan ketimpangan biaya perjalanan. Variabel Pclass dan Survived juga terlihat sebagai fitur kategorikal numerik dengan nilai 0–1 dan 1–3. Statistik ini membantu kita untuk memahami sebaran, pusat data, serta potensi *outlier* sebelum dianalisis lebih lanjut.

9. Mengecek *missing values* dan *duplicates*



```
df.isnull().sum()
```

	0
PassengerId	0
Survived	0
Pclass	0
Name	0
Sex	0
Age	177
SibSp	0
Parch	0
Ticket	0
Fare	0
Cabin	687
Embarked	2

dtype: int64

Gambar 2.4.53 Cek Missing Value

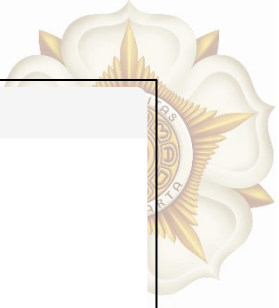
Pengecekan *missing values* di atas menunjukkan bahwa sebagian besar kolom tidak memiliki data kosong, namun terdapat 177 *missing* pada kolom *Age*, 687 *missing* pada *Cabin*, dan 2 *missing* pada *Embarked*. Kolom *Cabin*, *Age* dan *Embarked* perlu di-*impute* agar dataset tetap dapat digunakan.

```
# Duplicate check
df.duplicated().sum()
df.drop_duplicates(inplace=True)
```

Gambar 2.4.54 Cek Data Duplikat

Kode di atas digunakan untuk mengecek apakah terdapat baris duplikat dalam dataset. Jika ditemukan duplikasi, `df.drop_duplicates(inplace=True)` akan menghapus baris-baris tersebut.

10. Imputation



```
#imputation. kita isi nilai kosong
# kolom numerik
df['Age'].fillna(df['Age'].median(), inplace=True)
#df['Age'].fillna(df['Age'].mean(), inplace=True)

# kolom kategori
df['Cabin'].fillna(df['Cabin'].value_counts().index[0], inplace=True)
df['Embarked'].fillna(df['Embarked'].value_counts().index[0], inplace=True)
df.isnull().sum()
```

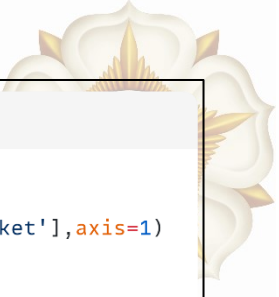
Gambar 2.4.55 Imputasi Missing Value

Kolom numerik seperti Age diisi menggunakan nilai median agar distribusinya tetap stabil, sedangkan kolom kategorikal seperti Cabin dan Embarked diisi menggunakan modus. Setelah dilakukan *imputation*, kita perlu memeriksa apakah masih ada *missing values* pada dataset.

	0
PassengerId	0
Survived	0
Pclass	0
Name	0
Sex	0
Age	0
SibSp	0
Parch	0
Ticket	0
Fare	0
Cabin	0
Embarked	0
dtype:	int64

Gambar 2.4.56 Missing Value

11. Memeriksa *categorical attribute*



```
# get X and y
df_X = df.drop(['PassengerId', 'Name', 'Survived', 'Cabin', 'Ticket'], axis=1)
df_y = df['Survived']

#check categorical attributes
cats = df_X.select_dtypes(include=['object', 'bool']).columns
print(cats)
```

Gambar 2.4.57 Memeriksa Atribut Kategorikal

Kode di atas memisahkan fitur (X) dan target (y) dengan menghapus kolom yang tidak diperlukan dari X, seperti PassengerId, Name, Survived, Cabin, dan Ticket. Selanjutnya, `df_X.select_dtypes(include=['object', 'bool']).columns` digunakan untuk mengecek atribut kategorikal dalam X. Hasilnya dapat dilihat pada Gambar Gambar 2.4.58.

```
Index(['Sex', 'Embarked'], dtype='object')
```

Gambar 2.4.58 Atribut Kategorikal

Output tersebut menunjukkan bahwa kolom kategorikal dalam fitur X adalah Sex dan Embarked, sehingga kedua kolom ini perlu dilakukan *encoding* sebelum *modelling*.

12. One hot encoding

```
# One Hot Encode
df_onehot = pd.get_dummies(df_X, columns=['Sex', 'Embarked'], drop_first=True)
df_onehot.head()
```

Gambar 2.4.59 One Hot Encoding

Kode di atas digunakan untuk *one-hot encoding* pada kolom kategorikal Sex dan Embarked menggunakan `pd.get_dummies()`. Parameter `drop_first=True` digunakan untuk menghindari *dummy trap* dengan menghapus salah satu kategori. Hasilnya, fitur kategorikal diubah menjadi kolom biner.



	Pclass	Age	SibSp	Parch	Fare	Sex_male	Embarked_Q	Embarked_S
0	3	22.0	1	0	7.2500	True	False	True
1	1	38.0	1	0	71.2833	False	False	False
2	3	26.0	0	0	7.9250	False	False	True
3	1	35.0	1	0	53.1000	False	False	True
4	3	35.0	0	0	8.0500	True	False	True

Gambar 2.4.60 Hasil One Hot Encoding

```

# ohe = OneHotEncoder(sparse_output=False, drop='first')
ohe = OneHotEncoder(sparse_output=False)

ohe.fit(df_X[['Sex', 'Embarked']])

# for col in column_categorical:
df_ohe = ohe.transform(df_X[['Sex', 'Embarked']])

# ohe.categories_
df_ohe

```

Gambar 2.4.61 One Hot Encoding dengan OneHotEncoder

Kode di atas menggunakan OneHotEncoder dari sklearn untuk melakukan one-hot encoding pada kolom Sex dan Embarked. Encoder di-fit pada data kategorikal, lalu ditransformasi menjadi representasi numerik biner. Hasil **df_ohe** berisi matriks encoded yang siap digabungkan ke fitur model.

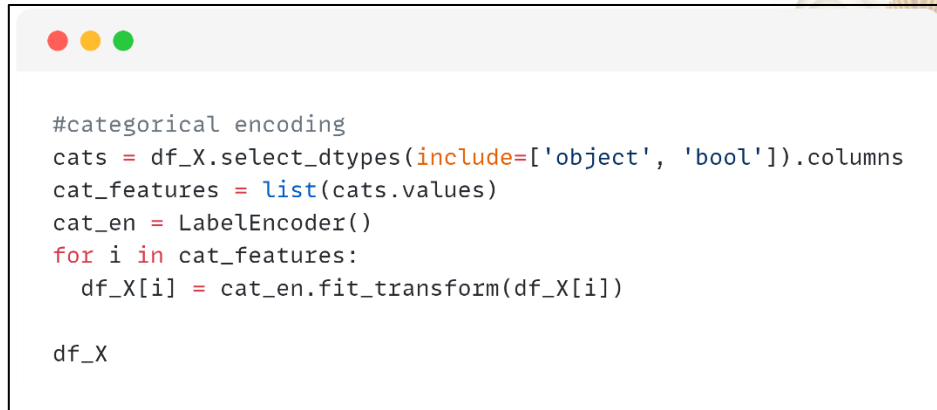
```

array([[0., 1., 0., 0., 1.],
       [1., 0., 1., 0., 0.],
       [1., 0., 0., 0., 1.],
       ...,
       [1., 0., 0., 0., 1.],
       [0., 1., 1., 0., 0.],
       [0., 1., 0., 1., 0.]])

```

Gambar 2.4.62 Hasil One Hot Encoding

13. Label encoding



```
#categorical encoding
cats = df_X.select_dtypes(include=['object', 'bool']).columns
cat_features = list(cats.values)
cat_en = LabelEncoder()
for i in cat_features:
    df_X[i] = cat_en.fit_transform(df_X[i])

df_X
```

Gambar 2.4.63 Label Encoding

Kode di atas digunakan untuk melakukan *label encoding* semua kolom kategorikal dalam X. Setiap kolom diubah menjadi nilai numerik menggunakan LabelEncoder, sehingga fitur kategorikal dapat digunakan oleh algoritma *machine learning* yang hanya menerima *input* numerik.

	Pclass	Sex	Age	SibSp	Parch	Fare	Embarked
0	3	1	22.0	1	0	7.2500	2
1	1	0	38.0	1	0	71.2833	0
2	3	0	26.0	0	0	7.9250	2
3	1	0	35.0	1	0	53.1000	2
4	3	1	35.0	0	0	8.0500	2
...	...	...	...	...	...	...	...
886	2	1	27.0	0	0	13.0000	2
887	1	0	19.0	0	0	30.0000	2
888	3	0	28.0	1	2	23.4500	2
889	1	1	26.0	0	0	30.0000	0
890	3	1	32.0	0	0	7.7500	1

891 rows × 7 columns

Gambar 2.4.64 Hasil Label Encoding

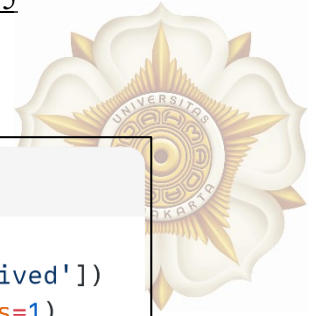
```
#label encoding for y
le = LabelEncoder()
le.fit(df_y)
df_y= le.fit_transform(df_y)
df_y
```

Gambar 2.4.65 Label Encoding untuk y

Kode di atas melakukan *label encoding* pada target y, mengubah nilai kategorikal seperti 0/1 atau label teks menjadi angka yang dapat digunakan oleh model *machine learning*.

[illegible]

Gambar 2.4.66 Hasil Label Encoding



14. Mengecek korelasi atribut

```
df_y_new = pd.DataFrame(df_y, columns=['Survived'])
df_gabung = pd.concat([df_X, df_y_new], axis=1)
df_gabung.corr()
```

Gambar 2.4.67 Cek Korelasi Atribut

Kode di atas menggabungkan kembali fitur (X) dan target (y) menjadi satu *DataFrame* lalu menghitung korelasi antar semua variabel untuk melihat hubungan fitur terhadap *Survived*.

	Pclass	Sex	Age	SibSp	Parch	Fare	Embarked	Survived
Pclass	1.000000	0.131900	-0.339898	0.083081	0.018443	-0.549500	0.162098	-0.338481
Sex	0.131900	1.000000	0.081163	-0.114631	-0.245489	-0.182333	0.108262	-0.543351
Age	-0.339898	0.081163	1.000000	-0.233296	-0.172482	0.096688	-0.018754	-0.064910
SibSp	0.083081	-0.114631	-0.233296	1.000000	0.414838	0.159651	0.068230	-0.035322
Parch	0.018443	-0.245489	-0.172482	0.414838	1.000000	0.216225	0.039798	0.081629
Fare	-0.549500	-0.182333	0.096688	0.159651	0.216225	1.000000	-0.224719	0.257307
Embarked	0.162098	0.108262	-0.018754	0.068230	0.039798	-0.224719	1.000000	-0.167675
Survived	-0.338481	-0.543351	-0.064910	-0.035322	0.081629	0.257307	-0.167675	1.000000

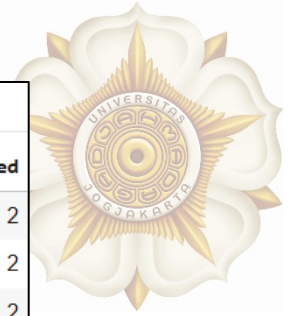
Gambar 2.4.68 Korelasi Antar Variabel

15. Membagi dataset

```
#hold out, dibagi menjadi training dan testing set
X_train, X_test, y_train, y_test = train_test_split(df_X, df_y, test_size=0.3, random_state=42)
X_train
```

Gambar 2.4.69 Pembagian Dataset

Kode di atas membagi dataset menjadi *training set* (70%) dan *testing set* (30%) menggunakan *train_test_split*. *Training* dipakai untuk melatih model, sedangkan *testing* untuk mengevaluasi performanya agar tidak *overfitting*.



X_train								
	Pclass	Sex	Age	SibSp	Parch	Fare	Embarked	
445	1	1	4.0	0	2	81.8583	2	
650	3	1	28.0	0	0	7.8958	2	
172	3	0	1.0	1	1	11.1333	2	
450	2	1	36.0	1	2	27.7500	2	
314	2	1	43.0	1	1	26.2500	2	
...	...	...	...	...	...	...	...	...
106	3	0	21.0	0	0	7.6500	2	
270	1	1	28.0	0	0	31.0000	2	
860	3	1	41.0	2	0	14.1083	2	
435	1	0	14.0	1	2	120.0000	2	
102	1	1	21.0	0	1	77.2875	2	
623 rows × 7 columns								

Gambar 2.4.70 Sampel Data Training

16. Scaling

```
#scaling
scaler = StandardScaler()
scaler.fit(X_train)
X_train = scaler.transform(X_train)
X_test = scaler.transform(X_test)
X_train
```

Gambar 2.4.71 Scaling dengan StandardScaler

Kode di atas merupakan kode untuk melakukan *standardization* menggunakan StandardScaler. Hasilnya dapat dilihat pada Gambar 2.4.72.



```
X_train
array([[ -1.63788124,  0.72077194, -1.91971935, ...,  1.99885349,
         0.98099823,  0.57000481],
       [ 0.80326712,  0.72077194, -0.0772525 , ..., -0.47932706,
        -0.46963364,  0.57000481],
       [ 0.80326712, -1.38740139, -2.15002771, ...,  0.75976322,
        -0.40613632,  0.57000481],
       ...,
       [ 0.80326712,  0.72077194,  0.92075038, ..., -0.47932706,
        -0.34778742,  0.57000481],
       [-1.63788124, -1.38740139, -1.15202483, ...,  1.99885349,
        1.72907416,  0.57000481],
       [-1.63788124,  0.72077194, -0.61463866, ...,  0.75976322,
        0.8913508 ,  0.57000481]])
```

Gambar 2.4.72 Sampel Data Training Setelah Scaling

```
#scaling
scaler = MinMaxScaler()
scaler.fit(X_train)
X_train = scaler.transform(X_train)
X_test = scaler.transform(X_test)
X_train
```

Gambar 2.4.73 Scaling dengan MinMaxScaler

Kode di atas merupakan kode *normalization* menggunakan MinMaxScaler. Hasilnya dapat dilihat pada Gambar 2.4.74.

```
array([[0.      , 1.      , 0.04498618, ..., 0.33333333, 0.15977676,
        1.      ],
       [1.      , 1.      , 0.34656949, ..., 0.      , 0.01541158,
        1.      ],
       [1.      , 0.      , 0.00728826, ..., 0.16666667, 0.02173075,
        1.      ],
       ...,
       [1.      , 1.      , 0.50992712, ..., 0.      , 0.02753757,
        1.      ],
       [0.      , 0.      , 0.17064589, ..., 0.33333333, 0.2342244 ,
        1.      ],
       [0.      , 1.      , 0.25860769, ..., 0.16666667, 0.15085515,
        1.      ]])
```

Gambar 2.4.74 Sampel Data Training Setelah Scaling

2.5. TUGAS & ANALISIS

- **Exploratory Data Analysis**

Dataset: <https://www.kaggle.com/datasets/sakshigoyal7/credit-card-customers>



- 1) Silahkan membaca dataset diatas menggunakan *dataframe*!
- 2) Jelaskan apa tujuan dari penggunaan dataset ini!
- 3) Definisikan atribut mana yang menjadi input dan atribut mana yang menjadi output/class/label!
- 4) Berikan penjelasan singkat untuk setiap atribut/*variable* dari dataset tersebut

- **Data Preprocessing**

Dataset: <https://www.kaggle.com/datasets/amitabhajoy/bengaluru-house-price-data>

- 1) Silahkan membaca dataset diatas menggunakan *dataframe*!
- 2) Buatlah visualisasi histogram untuk beberapa variabel numerik, lalu jelaskan pola distribusi yang terlihat!
- 3) Hitung dan visualisasikan korelasi antar variabel, kemudian jelaskan hubungan yang ditemukan!
- 4) Lakukan *exploratory data analysis* (EDA) dengan membuat *descriptive statistics* serta melakukan pengecekan *missing value* dan duplikasi. Jelaskan tindakan apa yang perlu dilakukan jika keduanya ditemukan!
- 5) Lakukan *preprocessing* lanjutan berupa *encoding* variabel kategorikal (*one-hot & label encoding*), melakukan *train–test split* (80:20), serta menerapkan *standardization* atau *normalization* pada fitur numerik. Jelaskan alasan penggunaan masing-masing teknik!

2.6. REFERENSI

<https://www.python.org/doc/>

<https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.html>

<https://numpy.org/>

Han, J., Pei, J., & Tong, H. (2022). Data Mining: Concepts and Techniques (4th ed.). Morgan Kaufmann/Elsevier.